Department of Electronics and Communication Engineering, Nirma University, Ahmedabad, Gujarat

TESTING AND VERIFICATION OF DIGITAL CIRCUITS (3EC605ME24)

Assignment Report

DESIGN AND VERIFICATION OF AMBA-AHB

By: Palash Singh (22BEC081)

E-mail: 22bec081@nirmauni.ac.in

1. Abstract

This project focuses on the functional verification of the AMBA AHB-Lite protocol using the Universal Verification Methodology (UVM) within the Xilinx Vivado simulation environment. The verification environment was developed for a single master and multiple slave configuration supporting burst types such as single, incrementing, and wrapping. A SystemVerilog-based layered UVM testbench was designed, incorporating components like driver, sequencer, monitor, scoreboard, and agent. Assertions were written to enforce protocol compliance, while functional coverage metrics ensured comprehensive stimulus validation. The entire verification flow, including waveform analysis and simulation, was successfully executed using Vivado, demonstrating that robust UVM-based verification can be achieved even outside traditional EDA toolchains like Xcelium or Questa.

2. INTRODUCTION

2.1 Background and Motivation

With increasing integration of IP blocks in modern SoCs (System-on-Chips), reliable and standardized communication protocols have become essential. AMBA AHB-Lite (Advanced High-performance Bus – Lite) is a widely adopted on-chip interconnect protocol developed by ARM. It supports pipelined bus transactions with high throughput while offering simplicity for single-master systems.

However, even with well-defined protocols, design bugs are common due to the complexity of burst transfers, arbitration, wait states, and timing dependencies. Functional verification becomes critical to validate that the system behaves according to the protocol specification under all valid and edge-case scenarios.

2.2 Why UVM for Verification

The Universal Verification Methodology (UVM) is an open-standard SystemVerilog framework designed to improve verification quality, reuse, and productivity. UVM enables the creation of reusable verification components like drivers, monitors, and scoreboards, which significantly reduce manual effort when verifying complex protocols like AHB-Lite.

Traditionally, UVM testbenches are developed and executed in advanced simulators like Cadence Xcelium or Mentor Questa. However, this project successfully implements UVM-based verification within the **Vivado simulation environment**, which is less commonly used for UVM but offers a powerful and accessible platform, especially in academic settings. This showcases that even within Vivado, high-quality verification using UVM, assertions, and coverage is achievable.

2.3 Objective

The primary objective of this project is to design a functional verification environment for the AMBA AHB-Lite protocol using UVM in Vivado. Specifically, the project aims to:

- Develop Verilog RTL for a single-master, multi-slave AHB-Lite system supporting **single, incrementing, and wrapping burst transfers**
- Create a layered UVM testbench including sequencer, driver, monitor, and scoreboard

- Verify protocol correctness using **SystemVerilog assertions**
- Ensure test coverage using **functional coverage models**
- Run and debug the complete simulation and waveform analysis within Vivado

3. AHB-Lite Protocol Overview

The AMBA AHB-Lite protocol is a simplified subset of the AMBA AHB (Advanced High-performance Bus) protocol, designed primarily for single-master systems. It offers high-performance, pipelined data transfer capabilities between a master and multiple slaves, making it ideal for embedded processors, memory interfaces, and peripheral controllers.

3.1 Key Features

- **Single master only**: AHB-Lite supports only one master, eliminating the need for arbitration logic.
- **Pipelined operation**: The address and control phase of a transfer overlaps with the data phase of the previous transfer, improving throughput.
- **Burst transfers**: Supports various burst types like **single**, **incrementing**, and **wrapping**.
- **Wait states**: Slaves can insert wait states using the HREADY signal.
- **Error response**: Slaves can signal errors using the HRESP signal.

3.2 Signal Description

Signal Name	Direction	Width	Description
clk	Input	1	System clock
rst	Input	1	Active-high synchronous reset
start	Input	1	Trigger signal to initiate a transaction
i_haddr	Input	32	Address of the transaction from master
i_hwrite	Input	1	Write enable: 1 = write, 0 = read
i_hsize	Input	3	Transfer size (byte/halfword/word)
i_hwdata	Input	32	Write data from master to slave
i_hburst	Input	3	Burst type: single/increment/wrap
o_hrdata	Output (wire)	32	Read data from slave to master
m_hready	Internal wire	1	Indicates if slave is ready for next transfer

3.3 Transfer Types

The HTRANS signal indicates the type of the current transfer:

- IDLE: No transfer
- NONSEQ: First transfer of a burst
- SEQ: Continuation of a burst

3.4 Burst Types

Burst Type	HBURST Value	Description
SINGLE	3'b000	Single transfer
INCR	3'b001	Sequential burst with linear increment
WRAP4	3'b010	4-beat burst with address wrapping
INCR4	3'b011	4-beat linear increment burst
Others	Assigned appropriate Values in the design	WRAP8, INCR8, WRAP16, etc. (optional in AHB-Lite)

4. Testbench Architecture

4.1 Interface and DUT Connectivity

The testbench interacts with the DUT using a SystemVerilog interface called ahb_top_i. This interface encapsulates all necessary signals such as clk, rst, i_haddr, i_hwdata, i_hwrite, i_hsize, i_hburst, and o_hrdata. Inside the testbench, this interface is accessed using a virtual interface, which allows UVM components like the driver and monitor to control or observe the DUT signals without hard-wiring them.

4.2 Sequence and Sequencer

The sequencer is responsible for generating high-level transaction objects that define the behavior of each AHB-Lite transfer. Sequences such as single reads, single writes, incrementing bursts, and wrapping bursts along with error sequences are modeled using custom uvm_sequence classes. These sequences are randomized or written explicitly to simulate a wide variety of scenarios.

4.3 Driver

The driver converts the high-level sequence items into pin-level signal activity. It drives the interface signals according to AHB-Lite timing and handshake rules. This includes asserting valid address, control, and data signals during each transfer, as well as waiting for HREADY to proceed with the next transaction. The driver ensures proper pipelining behavior by aligning address and data phases across clock cycles.

4.4 Monitor

The monitor passively observes signal activity on the interface. It listens to the DUT's inputs and outputs and reconstructs meaningful transactions from the pin-level signal changes. These reconstructed transactions are sent to both the scoreboard and coverage components. The monitor does not influence the DUT directly but plays a critical role in validation and analysis.

4.5 Scoreboard

The scoreboard checks functional correctness by comparing the expected results of a transaction with the actual outputs observed on the bus. For read transactions, it compares the HRDATA value against the expected data. For write transactions, it confirms that the correct address and data combination was applied. This helps identify protocol mismatches, data corruption, or logic bugs in the DUT.

4.6 Agent

The agent combines the sequencer, driver, and monitor into a single reusable verification unit. It can operate in active mode (generating and driving transactions) or passive mode (just monitoring). This modularity allows the same agent to be reused for both master and slave roles in extended testbenches. In this project, one active agent was used to drive and monitor the master-side transactions.

4.7 Environment

The environment is the top-level container that brings together the agent and scoreboard. It is responsible for configuring the components, establishing connections, and coordinating communication between them. It also acts as the central point for collecting functional coverage and protocol assertions, ensuring the testbench is scalable and maintainable.

4.8 Test and Simulation Control

The top-level UVM test class instantiates the environment and starts the testbench. It assigns the virtual interface, sets configuration parameters, and launches the appropriate sequence. This is the entry point for the simulation and can be customized to run different test scenarios such as basic reads, writes, burst operations, or error conditions.

5. Assertions

A separate module named assertion_check was created to validate protocol-level correctness using SystemVerilog Assertions (SVA). This module connects to the main DUT signals and ensures AHB-Lite protocol rules are enforced during simulation.

A total of **40 assertions** were implemented, categorized across the following aspects:

5.1 Functional Protocol Checks

Assertions verify reset conditions, signal behavior during transfers, valid transaction encoding, and write-read alignment. Examples include checks to ensure:

- Address and data are held stable during wait states
- Write data is driven only when i_hwrite is high
- o_hrdata is zero when start is low or reset is active

5.2 Burst and Size Validation

Assertions also check:

- Correct number of beats for each burst type (INCR, WRAP4, INCR8, etc.)
- Address increment according to i_hsize
- Address stability on invalid or aborted bursts

5.3 Pipelining and Wait-State Behavior

Assertions ensure proper pipelining by validating that the master:

- Holds values when HREADY is low
- Only proceeds to next transaction after the current one completes

These assertions were compiled and simulated within Vivado. Failing any assertion produces an error message during simulation, helping identify protocol bugs early.

---

7. Coverage

To measure the completeness of verification, a dedicated SystemVerilog **covergroup** named cia was written inside the monitor. This covergroup sampled transaction-level signals from the interface and ensured that all meaningful combinations of address, burst type, size, and data were exercised during simulation. The covergroup was triggered on every positive clock edge while reset was active.

6.1 Coverpoints

The following coverpoints were defined to track activity on protocol-relevant fields:

- start and rst signals to ensure testbench initialization and transaction triggering.
- i_haddr to track address range usage within [0:255].
- i_hwrite to distinguish between read and write operations.
- i_hsize to capture byte, half-word, and word transfers.
- i_hwdata to capture low and high data patterns.
- i_hburst to record all AHB burst modes including single, incrementing, and wrapping bursts.
- m_hready to observe wait-state behavior.
- o_hrdata to confirm read response activity

6.2 Cross Coverage

Several cross coverage scenarios were created to ensure:

- Burst types are exercised only when start is high and the slave is ready.
- Illegal address accesses are filtered based on start and m_hready.
- Correct timing between valid transactions and handshake conditions (start, m_hready).
- Distinction between legal and illegal transaction attempts.

Crosses like cross_illegal_burst, cross_legal_burst, cross_illegal_tx, and cross_legal_tx provide deep insight into edge-case coverage and help verify robustness against invalid stimulus.

6.3 Sampling and Integration

The coverage was sampled in the monitor during the run_phase. A transaction object tr captured the necessary fields from the interface, and cia.sample() was called only when reset was active. This ensured only meaningful activity was recorded.

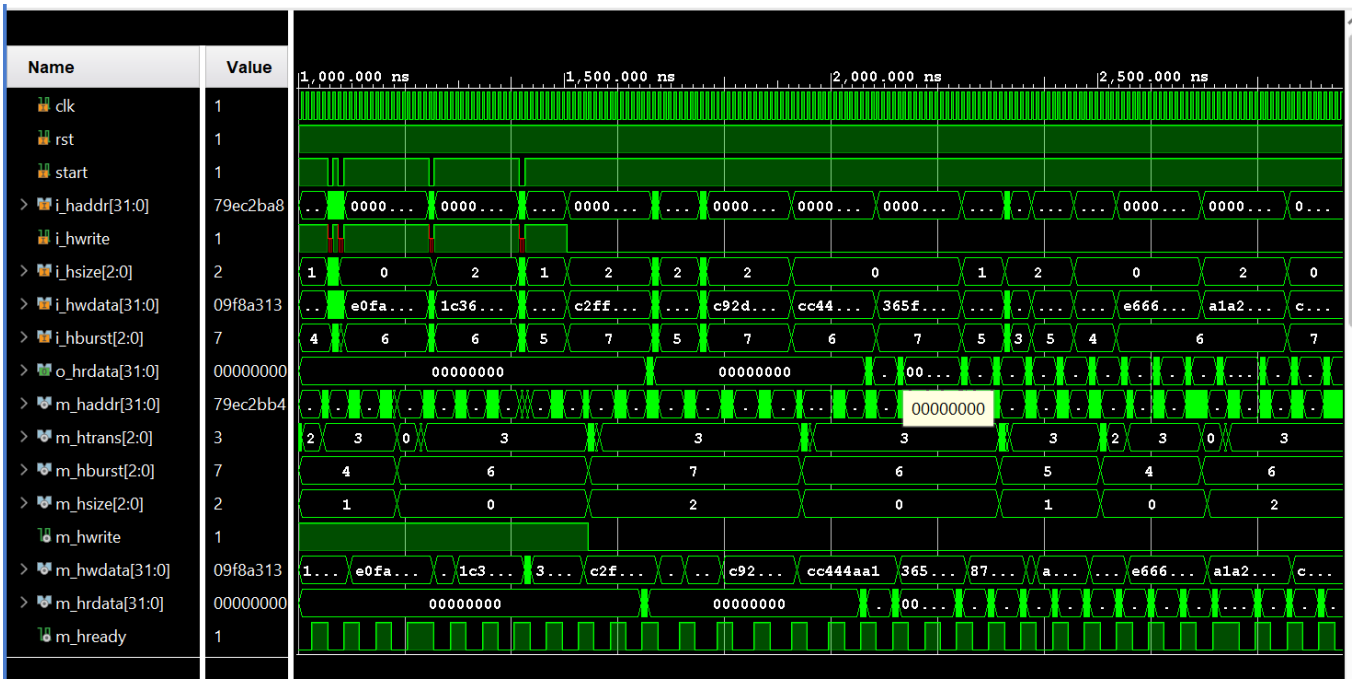
Overall, this coverage model provided confidence that the AHB-Lite protocol was exercised across functional, boundary, and illegal input conditions.

---

7. Results

The AHB-Lite bus design was successfully verified using a UVM-based testbench within the Vivado simulation environment. The results confirm both functional correctness and protocol compliance across various test scenarios and design configurations.

7.1 Functional Verification Results



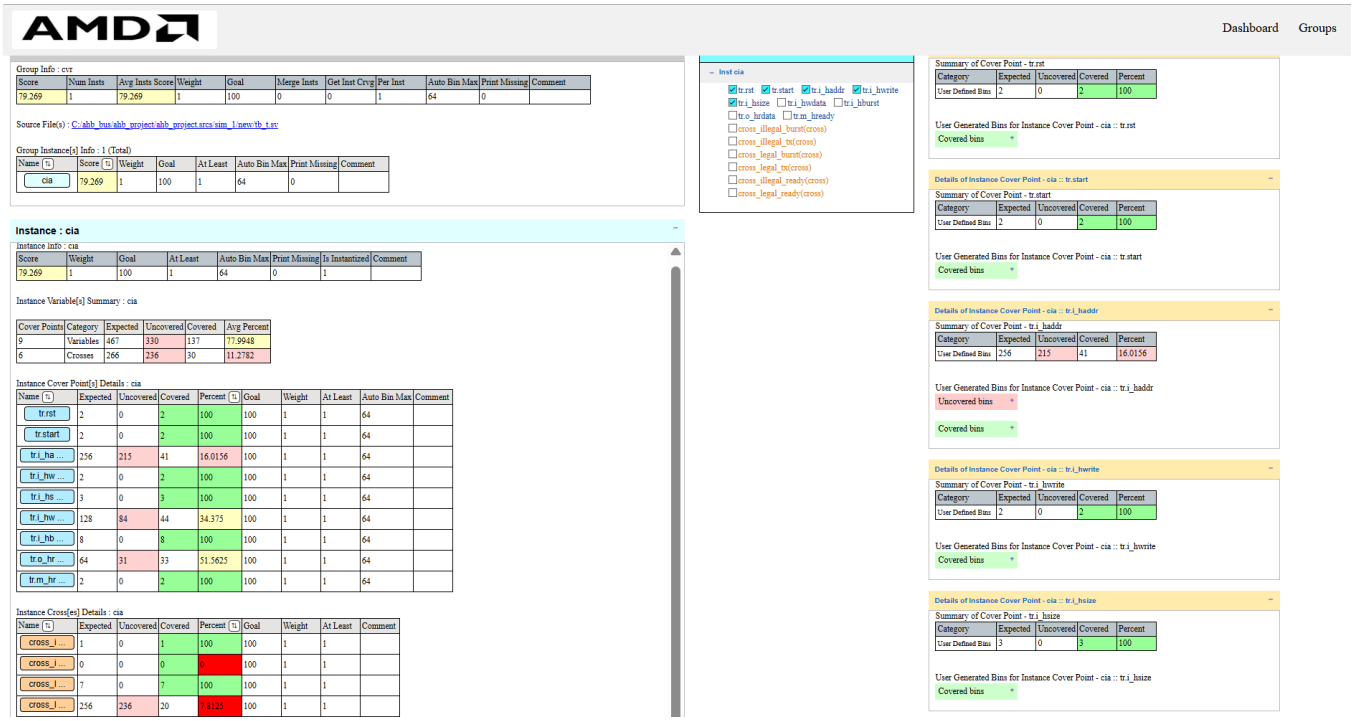
7.2 Assertion Results

```
Time: 75 ns  Iteration: 2  Process: /tb/dut/assert_inst//tb/dut,
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(414)
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(462)
Info: succ at time 85000
Time: 85 ns  Iteration: 2  Process: /tb/dut/assert_inst//tb/dut,
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(414)
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(462)
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(414)
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(462)
Info: succ at time 105000
Time: 105 ns  Iteration: 2  Process: /tb/dut/assert_inst//tb/dut
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(414)
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(462)
Info: succ at time 115000
Time: 115 ns  Iteration: 2  Process: /tb/dut/assert_inst//tb/dut
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(414)
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(462)
Info: succ at time 125000
Time: 125 ns  Iteration: 2  Process: /tb/dut/assert_inst//tb/dut
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(414)
UVM_INFO C:/ahb_bus/AHB_Lite/AHB_Lite.srcs/sim_1/new/tb.sv(462)
```

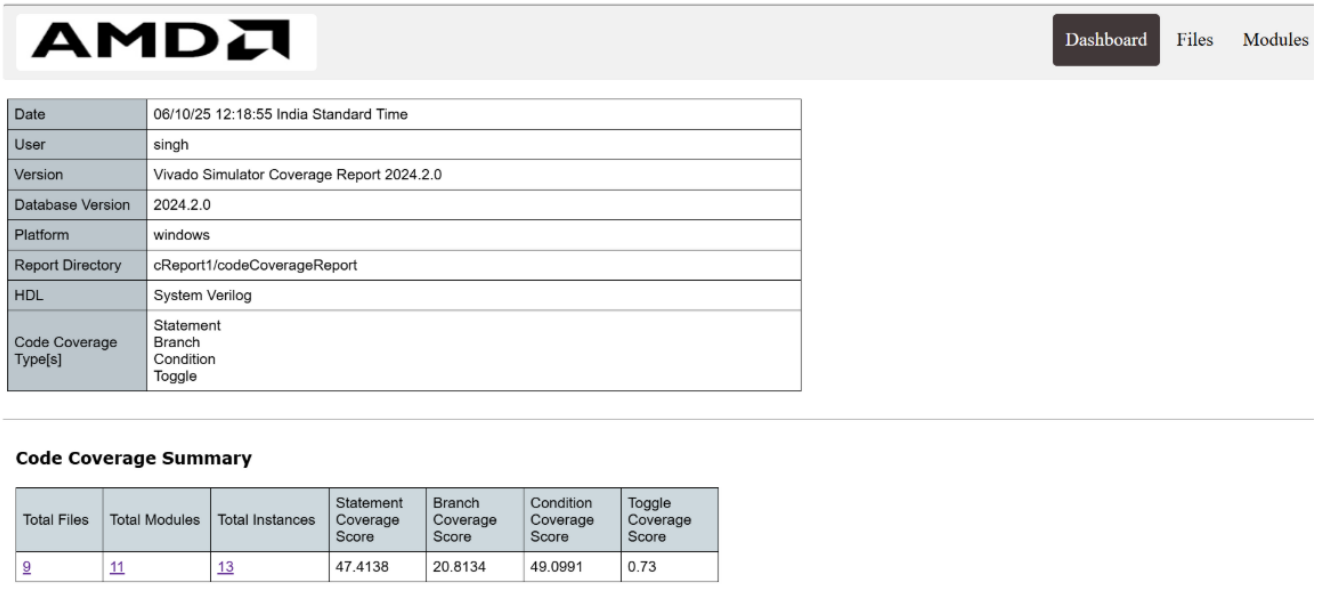
Note: It is not possible to attach results of all assertions this is just to give a glimpse of how output will look like

---

7.3 Coverage Results

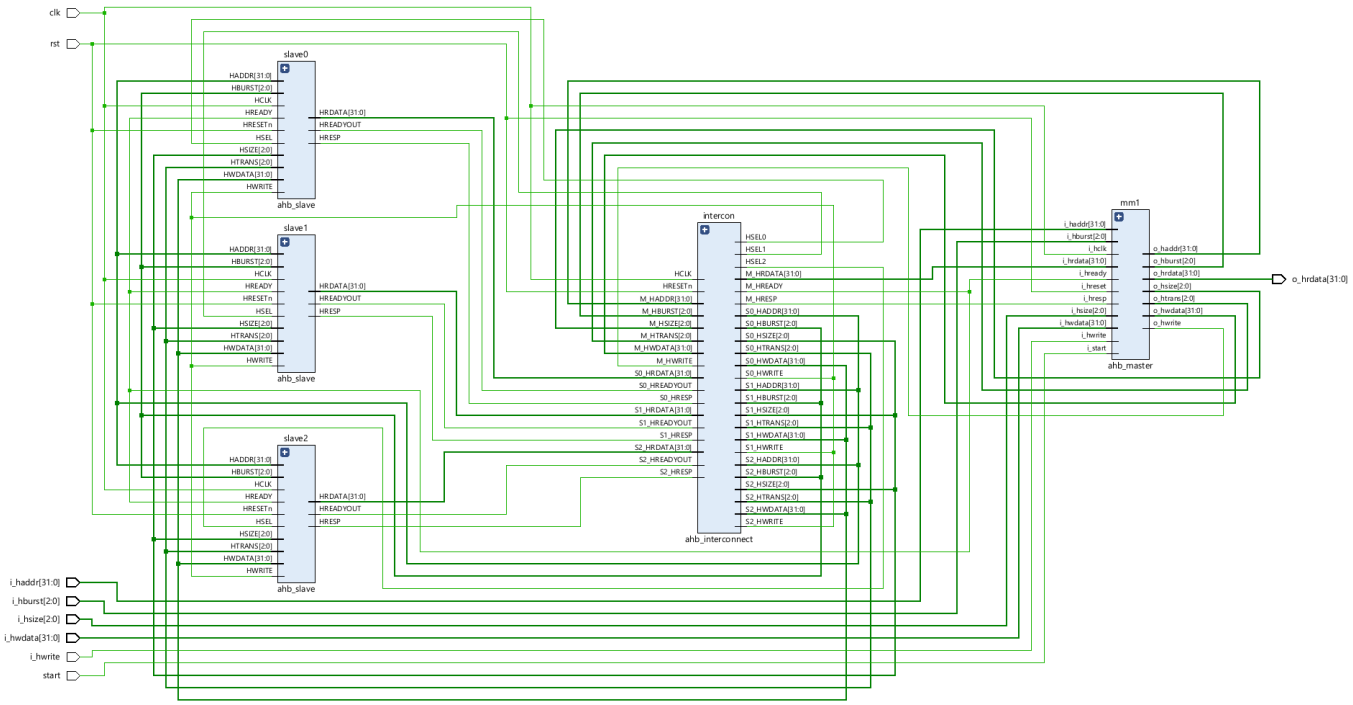


.



8. RTL AND DEVICE REPORTS

8.1 RTL

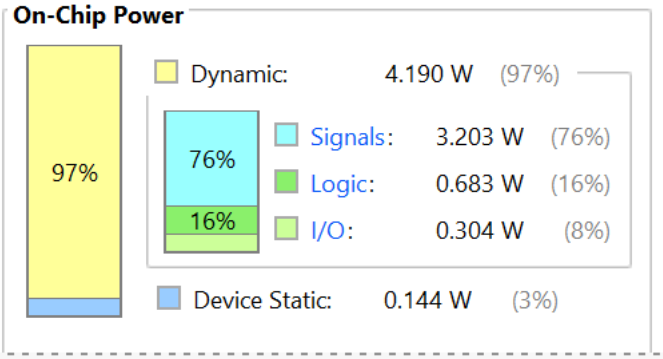


8.2 POWER REPORT

Summary

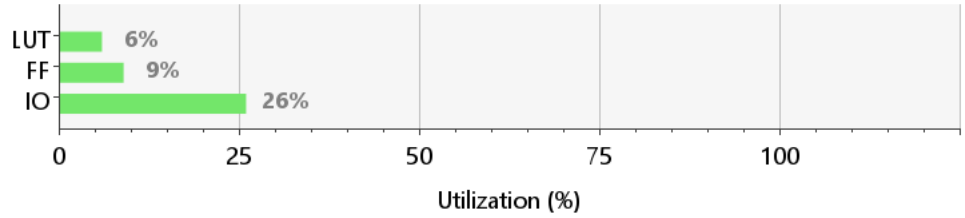
Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**4.334 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**33.1°C**



8.3 UTILIZATION

Resource	Utilization	Available	Utilization %
LUT	7806	134600	5.80
FF	25333	269200	9.41
IO	106	400	26.50



9. Conclusion

This project successfully demonstrated the functional verification of an AHB-Lite system using a UVM-based testbench implemented within Vivado. The verification environment included a layered architecture with a sequencer, driver, monitor, scoreboard, and assertions, all driven through a virtual interface.

Multiple test scenarios were executed to validate single transfers, burst operations, wait-state behavior, and error handling. Protocol correctness was enforced through a dedicated SystemVerilog assertion module, which included 40 unique properties covering both functional and timing aspects. All assertions passed without violation.

Functional coverage was collected using a custom covergroup embedded in the monitor. It confirmed that all valid transaction types, data widths, and burst modes were exercised. Cross coverage further ensured robustness by validating illegal and edge-case behaviors.

The overall verification flow confirmed the design's correctness and compliance with the AHB-Lite specification. The use of UVM, assertions, and coverage metrics ensured that the testbench was both scalable and reusable for future AMBA-based designs.

10. References

1. **AMBA AHB-Lite Protocol Specification**
   ARM Limited, AMBA Specification v2.0, https://developer.arm.com
2. **Universal Verification Methodology (UVM) User Guide**
   Accellera Systems Initiative, https://accellera.org/downloads/standards/uvm

3. **SystemVerilog Assertions Handbook**
   Ben Cohen, Srinivasan Venkataramanan, et al.
   "Using SystemVerilog for Formal and Assertion-Based Verification", 3rd Edition.

4. **Vivado Design Suite User Guide: Logic Simulation (UG900)**
   Xilinx, 2023
   https://www.xilinx.com/support/documentation-navigation/design-hubs/dh0002-vivado-design-suite.html

5. **SystemVerilog for Verification: A Guide to Learning the Testbench Language Features**
   Chris Spear, Greg Tumbush, Springer, 3rd Edition.

6. **Digital Design and Computer Architecture**
   David Harris & Sarah Harris, 2nd Edition – Morgan Kaufmann.

7. **AHB Bus Functional Model (BFM) Examples and Tutorials**
   Various online forums and documentation (EDA Playground, GitHub, StackExchange).